Exercise 9 - Shared Pointers and Vectors

This weeks exercise will focus on implementing key data structures from the `std_library`.
This includes `std::shared_ptr<datatyp>`, another typ of `smart pointer`, as well as `std::vector<datatyp>`, which is propably the most important data structure in all of `C++`.

If you have any questions or problems, please write in the Moodle!

Exercise 1 — Shared Pointers

For this Exercise try out your code in `main.cpp`.

a) Define a variable `int x = 42` and two `std::shared_ptr`, the first one pointing to x (use std::make_unique()) and the second one being equal the first pointer. What will the method `use_count()` print for the first pointer?

b) Make the second pointer a `nullptr`. What will get printed now, when calling `use_count()` on both pointers? Explain the result from `a & b` with respect to the concept of a shared pointer. What does `use_count()` return?

c) Print the address of the first pointer as well as the result of the `get()` method of that same pointer. Are the values different?
Explain what `get()` does with respect to the idea of `smart_ptrs` being `wrapper classes` for `raw pointers`.

Exercise 2 — Vectors

For this Exercise try out your code in `main.cpp`.

a) Define a `std_vector<int>` filled with the values from `0` to `9`. Erase the 5th `Element` of the vector. Afterwards, delete the first five `Elements` of the vector.

You can use the `erase()` method for both operations.
Use `<vector>.begin()` to `get` the first `Element` of the vector.

b) Write a function `printVector(std::vector<int> my_vector)` iterating over the passed vector and printing every `Element`.

You can use `<vector>.size()` to get the current length of a vector to use as upper bound for your `loop`.